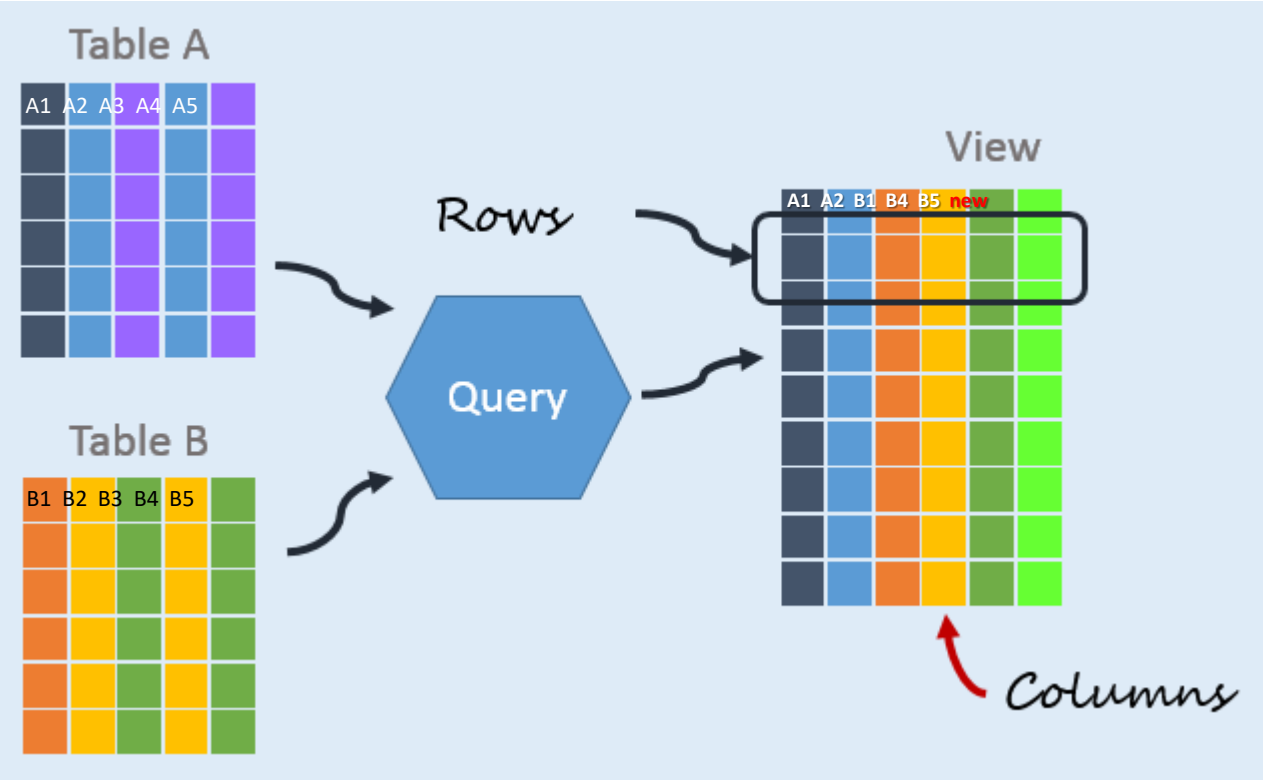
L9: View and Index

Sirawich.V

Ref. https://www.mysqltutorial.org/

9.1 View

View is a searchable object or a virtual table defined by a SELECT query.



Modified from https://www.essentialsql.com/wp-content/uploads/2014/05/AnatomyOfAView.png

Advantage

- **Hide complicated query**
- Provide **extra security** layer to restrict access to certain column to specific users
- Enable **query reuse** across different parts of an application or among multiple users.

Disadvantage

- Limited Updateability
- Data Redundancy
- Cannot be associated with trigger

View vs Temporary Table

- A temporary table is a real table that temporarily stores data for the duration of a session or transaction. Once the operation or session is complete, the data in the temporary table is lost.

- A view is a predefined SELECT query that operates on existing data without duplicating it. It can be called by another section.

CREATE VIEW

Syntax

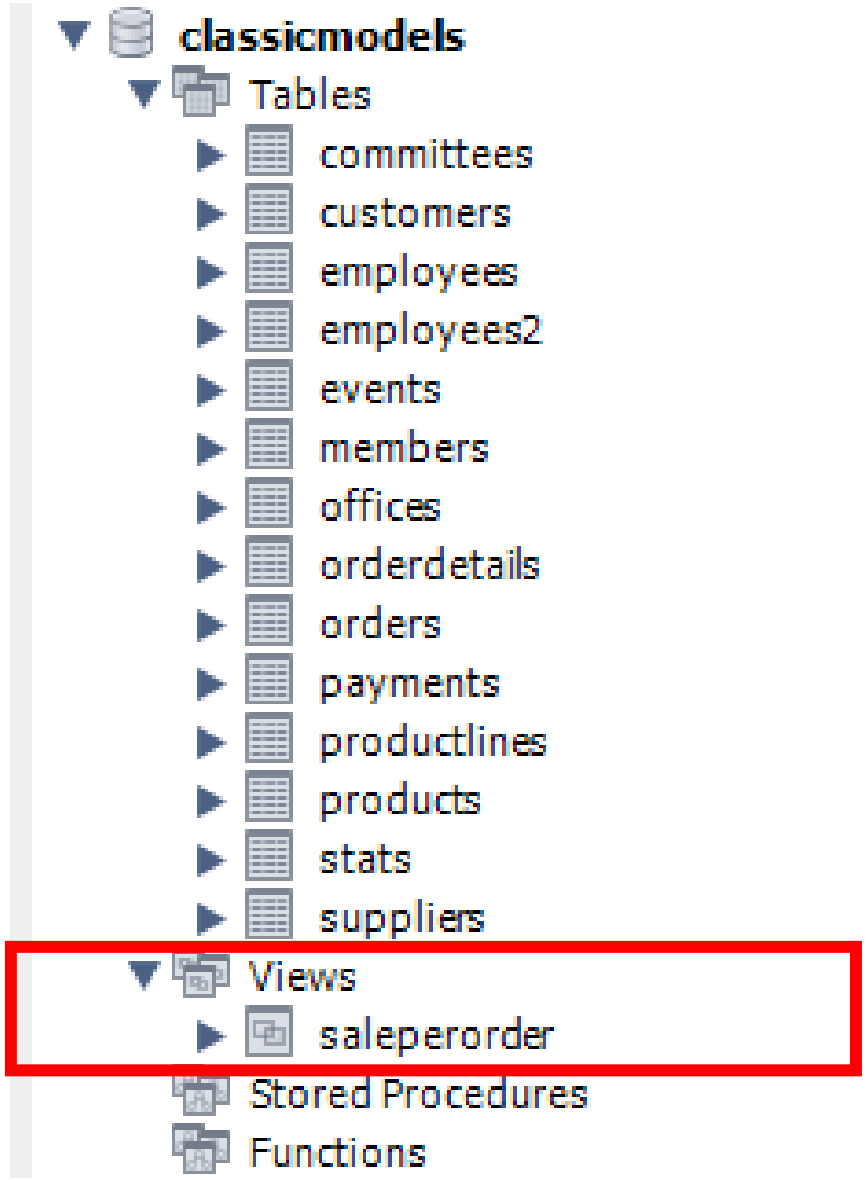
```
CREATE VIEW view_name [(column_list)]
AS
  SELECT-statement;
```

Example

```
CREATE VIEW salePerOrder AS
    SELECT
        orderNumber,
        SUM(quantityOrdered * priceEach) total
    FROM
        orderDetails
    GROUP by orderNumber
    ORDER BY total DESC;
```

CREATE VIEW

Tables_in_classicmodels	Table_type
employees	BASE TABLE
employees2	BASE TABLE
events	BASE TABLE
members	BASE TABLE
offices	BASE TABLE
orderdetails	BASE TABLE
orders	BASE TABLE
payments	BASE TABLE
productlines	BASE TABLE
products	BASE TABLE
saleperorder	VIEW
stats	BASE TABLE



Process VIEW

Alter

```
ALTER VIEW recent_orders AS
SELECT
    order_id,
    customer_name,
    order_date,
    total_amount,
    order_status
FROM orders
WHERE
    order_date >= DATE_SUB(NOW(), INTERVAL 30 DAY);
```

Drop

```
DROP VIEW recent_orders;
```

Get Information

```
SHOW CREATE VIEW recent_orders;
```

9.2 Index

An index is a data structure such as a B-Tree that improves the speed of data retrieval on a table at the cost of additional writes and storage to maintain it.

Without an index, MySQL must begin with the first row and then read through the entire table to find the relevant rows. The larger the table, the more this costs.

CREATE INDEX

When you create a table with a primary key or unique key, MySQL automatically creates a special index named PRIMARY.

Syntax: with CREATE TABLE

```
CREATE TABLE t(
    c1 INT PRIMARY KEY,
    c2 INT NOT NULL,
    c3 INT NOT NULL,
    c4 VARCHAR(10),
    INDEX (c2,c3)
);
```

Syntax: CREATE INDEX

```
CREATE INDEX
    index_name
ON
    table_name (column_list);
```

DROP INDEX

Syntax

```
DROP INDEX index_name ON table_name;
```

SHOW INDEX

Syntax

```
SHOW INDEXES FROM table_name;
```

Drop Index: https://www.mysqltutorial.org/mysql-index/mysql-drop-index/
Show Index: https://www.mysqltutorial.org/mysql-index/mysql-show-indexes/

Index vs Non-Index Query

1 million rows

id	name	name_with_index
1	ABC_000001	ABC_000001
2	ABC_000002	ABC_000002
3	ABC_000003	ABC_000003
4	ABC_000004	ABC_000004
5	ABC_000005	ABC_000005
6	ABC_000006	ABC_000006

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_t
employees	0	PRIMARY	1	id	A	998360	NULL	NULL		BTREE
employees	1	idx_name	1	name_with_index	A	999050	NULL	NULL	YES	BTREE

```
mysql>
select * from employees where name_with_index =
"ABC_000006" ;
+----+------------+-----------------+
| id | name       | name_with_index |
+----+------------+-----------------+
|  6 | ABC_000006 | ABC_000006      |
+----+------------+-----------------+
1 row in set (0.00 sec)
```

```
mysql>
select * from employees where name =
"ABC_000006";
+----+------------+-----------------+
| id | name       | name_with_index |
+----+------------+-----------------+
|  6 | ABC_000006 | ABC_000006      |
+----+------------+-----------------+
1 row in set (0.22 sec)
```

Index vs Non-Index Query

- Tables that **should have** index:
  - Large tables
  - Tables that are frequently retrieved for a set of queries.
- Attributes that **should** be indexed:
  - Attributes used for joining (JOIN conditions)
  - Attributes used for sorting (in ORDER BY clause)
  - Attributes used for grouping (in GROUP BY clause)
  - Attributes used in aggregation functions
  - Attributes used in a WHERE clause
  - Attributes used as a foreign key

- Tables that **should NOT** have index:
  - Small tables
  - Tables that are frequently manipulated for a set of queries.
- Attributes that **should NOT** be indexed
  - Attributes with type image, bit and text
  - Attributes with small domain (e.g., gender)
  - Attributes with large size (e.g., char(100))
  - Attributes that are rarely used in any query